

Handling Missing Data

What's Covered

1. Quick Recap — Types of Missingness (MCAR, MAR, MNAR)
2. Complete Case Analysis (CCA) — In Depth
3. Arbitrary Value Imputation
4. End-of-Tail Imputation
5. Frequent Category Imputation
6. Random Sample Imputation
7. Missing Indicator — Adding the Flag
8. KNN Imputation — Revisited with sklearn
9. Multivariate Imputation (MICE / IterativeImputer)
10. Comparing All Strategies — When to Use What
11. Feature Engineering Perspective on Missing Data
12. Common Pitfalls
13. Summary

1. Quick Recap — Types of Missingness

Type	What It Means	Safe Strategy
MCAR	Missing Completely At Random — no pattern	Any technique; deletion is safe
MAR	Missing At Random — depends on other observed features	Imputation using other features (KNN, MICE)
MNAR	Missing Not At Random — depends on the missing value itself	Imputation + MissingIndicator; be cautious

Day 35 goes deeper into each individual strategy — understanding exactly when and why each one works, and how to implement it correctly in a production feature engineering pipeline.

2. Complete Case Analysis (CCA)

Definition: Remove ALL rows that have at least one missing value. Only keep rows where every single column is present — the 'complete cases'.

2.1 When CCA is Valid

- Data is MCAR — missingness is truly random, unrelated to any variable.
- Missing data is a small fraction — typically less than 5%.
- Enough data remains after deletion — model still has sufficient examples.

2.2 Dangers of CCA

- If data is MAR or MNAR, deleting rows introduces systematic bias — the remaining data is no longer representative of the population.
- Significant loss of information when missingness is > 5%.
- In a dataset with many columns, even 5% missingness per column can eliminate a huge fraction of rows.

```
# Complete Case Analysis – drop ALL rows with any NaN
df_cca = df.dropna()

# How much data was retained?
retained_pct = len(df_cca) / len(df) * 100
print(f'Retained: {retained_pct:.1f}% of rows')

# CCA only on specific columns (not all)
df_cca2 = df.dropna(subset=['Age', 'Salary'])
```

Python

3. Arbitrary Value Imputation

Replace missing values with a fixed, arbitrary value that is deliberately chosen to be OUTSIDE the normal range of the feature — so the model can identify these rows as 'was missing' through the unusual value itself.

3.1 Common Arbitrary Values

- Numerical columns: 0, -1, -999, -9999, 999 (a value far from the normal range)
- Categorical columns: 'Missing', 'Unknown', 'None'

```
# Fill numerical missing values with -999 (signals 'was missing')
df['Age'].fillna(-999, inplace=True)

# Fill categorical missing values with 'Missing'
df['City'].fillna('Missing', inplace=True)

# Using SimpleImputer with strategy='constant'
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='constant', fill_value=-999)
X_train_imp = imp.fit_transform(X_train)
```

Python

Arbitrary Imputation	Details
Advantage	Simple and fast; model can learn that unusual value = 'was missing'
Disadvantage	Distorts feature distribution; can confuse models like KNN and Linear Regression
Best for	Tree-based models (Decision Tree, XGBoost) that split on thresholds
Avoid for	Linear models, KNN — they are sensitive to the magnitude of values

4. End-of-Tail Imputation

A smarter version of arbitrary imputation — instead of a completely random value like -999, use a value at the EXTREME END of the feature's actual distribution. This is less disruptive to the distribution while still flagging that something unusual happened.

4.1 How to Compute the Fill Value

```
# For normally distributed data: mean ± 3 standard deviations
upper_limit = df['Age'].mean() + 3 * df['Age'].std()
lower_limit = df['Age'].mean() - 3 * df['Age'].std()

# For skewed data: use IQR method
Q1 = df['Salary'].quantile(0.25)
Q3 = df['Salary'].quantile(0.75)
IQR = Q3 - Q1
upper_limit = Q3 + 1.5 * IQR

# Fill missing values with the computed extreme value
df['Salary'].fillna(upper_limit, inplace=True)
```

Python

End-of-tail imputation is more principled than arbitrary values because the fill value is derived from the actual data distribution. It pushes imputed values to the tail without inventing values that are completely impossible for the feature.

5. Frequent Category Imputation

For CATEGORICAL features, replace missing values with the MOST FREQUENT category (the mode). This is the categorical equivalent of mean/median imputation for numerical features.

```
# Manual: replace NaN with mode
mode_city = df['City'].mode()[0]
df['City'].fillna(mode_city, inplace=True)

# Using SimpleImputer with most_frequent strategy
from sklearn.impute import SimpleImputer
cat_imputer = SimpleImputer(strategy='most_frequent')
X_train[['City', 'Gender']] = cat_imputer.fit_transform(
    X_train[['City', 'Gender']])
```

Python

Frequent Category Imputation	Details
Advantage	Simple and fast; preserves the most common pattern in data
Disadvantage	Amplifies the majority class; can bias model towards the most common category
When valid	Data is MCAR; the majority category truly represents a reasonable default
When to avoid	MNAR — when missing values belong to a specific rare category

6. Random Sample Imputation

Instead of always filling with the same value (mean, median, mode), random sample imputation fills each missing value with a randomly chosen value drawn from the existing observed values of that feature.

6.1 Why It Helps

- Preserves the original distribution of the feature — mean, std, and shape remain unchanged after imputation.
- Avoids the 'spike at the mean' problem that simple imputation creates.
- Introduces randomness — adding natural variance instead of a single fixed value.

```
# Get all observed (non-missing) values from the column
observed = df['Age'].dropna()

# Count how many values are missing
n_missing = df['Age'].isnull().sum()

# Randomly sample that many values from the observed values
random_fill = observed.sample(n=n_missing, replace=True,
    random_state=42).values

# Fill the missing positions with these random samples
df.loc[df['Age'].isnull(), 'Age'] = random_fill
```

Python

Random Sample Imputation	Details
Advantage	Preserves distribution; introduces realistic variance; no artificial spike
Disadvantage	Non-deterministic — different random seeds give different results; hard to reproduce exactly
Best for	Exploratory analysis; when distribution preservation is critical
Limitation	Must store the observed values at training time to use at inference time

7. Missing Indicator — Adding the Flag

Add a new binary column (1 = was missing, 0 = was present) for each feature that had missing values. This is typically combined WITH another imputation strategy — not used alone.

The key insight: the pattern of missingness (who left this blank and why) is itself a signal. A missing income value might mean the person earns very high or very low — and that correlation can be captured by the indicator feature.

Python

```
# Method 1: create indicator column manually
df['Age_missing'] = df['Age'].isnull().astype(int)

# Then impute Age with any chosen strategy
df['Age'].fillna(df['Age'].median(), inplace=True)

# Method 2: add_indicator=True inside SimpleImputer
from sklearn.impute import SimpleImputer
imp = SimpleImputer(strategy='median', add_indicator=True)
X_imp = imp.fit_transform(X_train)
# Extra columns appended at the end for each feature with NaN

# Method 3: MissingIndicator separately
from sklearn.impute import MissingIndicator
indicator = MissingIndicator()
flags = indicator.fit_transform(X_train)
# flags is a binary matrix — one column per feature that had NaN
```

Best Practice: Always add a MissingIndicator when you suspect MAR or MNAR. The indicator lets the model learn whether being missing is itself predictive — something imputation alone can never capture.

8. KNN Imputation — Revisited

Finds the K rows most similar to the row with the missing value (based on other non-missing features) and fills the missing value with the weighted average of those K neighbours' values.

Python

```
from sklearn.impute import KNNImputer

# Create KNN imputer — n_neighbors controls how many similar rows to use
knn_imp = KNNImputer(n_neighbors=5, weights='uniform')

# Fit on training data — memorises observed values
knn_imp.fit(X_train)

# Transform training and test sets
X_train_imp = knn_imp.transform(X_train)
X_test_imp = knn_imp.transform(X_test)

# weights='distance' gives closer neighbours more influence
knn_dist = KNNImputer(n_neighbors=5, weights='distance')
```

KNNImputer	Details
Advantage	Leverages feature correlations for smarter imputation
Disadvantage	Slow on large datasets — $O(n^2)$ distance computation
n_neighbors	Smaller k = more local, noisier; larger k = smoother but slower
weights=uniform	All k neighbours contribute equally to the filled value
weights=distance	Closer neighbours have more influence — often more accurate
Prerequisite	All columns must be numerical — encode categoricals first

9. Multivariate Imputation — MICE / IterativeImputer

MICE (Multivariate Imputation by Chained Equations) is the most sophisticated imputation strategy. Each feature with missing values is treated as a regression target and modelled using all other features. This process repeats (iterates) until the imputed values converge.

9.1 How MICE Works — Step by Step

- Step 1: Fill all missing values with a simple initial estimate (e.g. mean).
- Step 2: For each column with missing values, treat it as the target variable and build a regression model using all other columns.
- Step 3: Predict the missing values using this model.
- Step 4: Repeat steps 2-3 for every column with missing values — one full pass = one iteration.
- Step 5: Repeat the full cycle until imputed values stabilise (converge).

```
# Enable experimental feature before importing
from sklearn.experimental import enable_iterative_imputer
from sklearn.impute import IterativeImputer

# Default estimator is BayesianRidge (linear regression)
mice_imp = IterativeImputer(max_iter=10, random_state=42)

# Use a different estimator — e.g. Random Forest for non-linear data
from sklearn.ensemble import RandomForestRegressor
mice_rf = IterativeImputer(
    estimator=RandomForestRegressor(n_estimators=10, random_state=0),
    max_iter=10, random_state=42)

# Fit and transform
X_train_imp = mice_imp.fit_transform(X_train)
X_test_imp = mice_imp.transform(X_test)
```

Python

IterativeImputer (MICE)	Details
Most accurate for	Datasets where features are strongly correlated
max_iter	Number of full imputation cycles — typically 10 is sufficient
Default estimator	BayesianRidge — fast and works well for most data
Custom estimator	RandomForestRegressor for non-linear relationships
Status in sklearn	Experimental — must import enable_iterative_imputer first
Slowest method	Most computationally expensive but most accurate

10. Comparing All Strategies — When to Use What

Strategy	Best When	Avoid When
CCA (deletion)	< 5% missing, MCAR, large dataset	> 5% missing or MAR/MNAR

Strategy	Best When	Avoid When
Mean imputation	Numerical, symmetric, no outliers, MCAR	Skewed data or outliers present
Median imputation	Numerical, skewed or has outliers	Normally distributed data (mean is better)
Mode imputation	Categorical columns, MCAR	MNAR — biases towards majority class
Arbitrary value	Tree-based models, signal needed	Linear models, KNN (scale-sensitive)
End-of-tail	When distribution shape must be preserved	When extreme values distort the model
Random sample	Distribution preservation critical	Production — reproducibility required
Missing Indicator	MAR or MNAR suspected — add alongside imputer	When missingness has no predictive value
KNN Imputation	Correlated features, small-medium dataset	Very large datasets (slow)
MICE / Iterative	Strongly correlated features, accuracy needed	Large datasets — computationally expensive

11. Feature Engineering Perspective on Missing Data

From a feature engineering standpoint, missing values are not just a problem to fix — they are a SOURCE OF INFORMATION. How you handle them can create new powerful features or silently destroy predictive signal.

- The FACT that a value is missing is often predictive. Always consider adding a MissingIndicator alongside any imputation.
- The pattern of which columns are missing together can reveal underlying subgroups in the data.
- Always impute AFTER splitting — the imputation statistics (mean, mode, distribution) must come from training data only.
- When comparing models: always test both with and without MissingIndicator features to see which performs better.
- In production: store the imputation statistics from training (mean, median, mode, distribution) so new data can be imputed consistently — a Pipeline does this automatically.

The golden rule: never decide on an imputation strategy without examining the missingness pattern first. The right strategy depends entirely on WHY the data is missing — not just how much is missing.

12. Common Pitfalls

- ■ **Imputing before train-test split:** Leakage — training data statistics are contaminated by the test set. Always split first.
- ■ **Using mean imputation on skewed or outlier-heavy data:** The mean is pulled by extremes. Use median instead.
- ■ **Ignoring the missing indicator:** If missingness carries signal (MAR/MNAR), discarding it loses predictive information.
- ■ **Using CCA when missingness is not MCAR:** Systematically removes certain types of rows, making data no longer representative.
- ■ **Using KNN or MICE on very large datasets without considering compute cost:** These methods scale poorly — use SimpleImputer for large datasets.
- ■ **Using arbitrary values with distance-based models:** Values like -999 distort distances in KNN and influence linear model coefficients badly.
- ■ **Not storing imputation statistics for production:** Always save the fitted imputer (via joblib) or use a Pipeline — never recompute imputation values on new data.

13. Summary

Strategy	Key Takeaway
CCA	Drop rows with NaN — only safe for < 5% MCAR data
Mean / Median / Mode	SimpleImputer — fast, simple; median for skewed, mode for categorical
Arbitrary Value	Fill with -999 or 'Missing' — good for tree models; signals missingness
End-of-Tail	Fill with $Q3 + 1.5 \times IQR$ or $\text{mean} + 3\sigma$ — principled extreme value
Random Sample	Fill from observed distribution — preserves shape; less reproducible
Missing Indicator	Add binary 1/0 flag — always combine with another imputation method
KNNImputer	Uses feature correlations — accurate but slow on large data
IterativeImputer	MICE — most accurate; fits a model per feature; computationally heavy
Fit rule	ALWAYS fit on training data only — never on the full dataset
Pipeline	Wrap imputer in Pipeline — prevents leakage in cross-validation
Production	Save the fitted Pipeline/imputer with joblib for inference

Notes by: Amol Jagtap | amoljagtap3001@gmail.com | Day 35 — Handling Missing Data (Advanced)